

## COM410 Programming in Practice

### Case Study

#### Design Summary

---

The implementation is built around 4 classes described as follows

##### Vehicle

A class representing a randomly generated vehicle. Each vehicle is represented as a string type (car/van/HGV/bus/minibus), a string colour (Gray/Black/White/Blue/Red/Silver/Green/Orange/Yellow/Bronze) and an integer value which is the number of points awarded for that type and colour combination. Getter methods enable other classes to retrieve the Vehicle properties.

The constructor generates a random vehicle according to the percentage distribution of vehicle types and colours and allocates the number of points according to the combination.

The class also implements a **toString()** method that returns a text representation of the car.

##### Player

A class representing an individual member of a team as a string name and an integer score.

The constructor creates a player object by assigning the name to a value provided and initialises the score to zero.

The class implements a **play()** method that simulates that player's turn by repeatedly generating random vehicles, printing the description of each and updating the player's score until a red vehicle is generated. When the player's turn is complete, their score is printed to the console.

The class also implements a **toString()** method that returns a text representation of the player and their score.

##### Team

A class representing a team of players, organised as an array. Each team also has a string name.

The constructor generates a team of players by prompting the user for a player name and accepting a string from the keyboard. The name is used in a call to the constructor for the Player object.

##### MotorwayCricket

The top-level class that contains the **main()** method that runs the game.

The game logic generates 2 teams and calls the `play()` method for each. Then, the scores for each team are reported and the team with the highest total is declared the winner.

The relationship between the classes is illustrated by the following diagram that shows how the `main()` method in the `MotorwayCricket` class creates the two `Team` objects. In creating the teams, the `Team` class creates three `Player` objects and simulates each player's turn by calling the `play()` method. The `Player play()` method generates a stream of random `Vehicle` objects, calculating the cumulative score and returning when the turn is ended by the generation of a red vehicle.

